

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Сибирский государственный университет телекоммуникаций и
информатики»

Кафедра вычислительных систем

Отчет по лабораторной работе №1
по дисциплине
Архитектура распределенных приложений

по теме:
НАЧАЛО РАБОТЫ С DOCKER

Студент:
Группа ИА-331

Я.А Гмыря

Предподаватель:
Преподаватель

Т.В Челканова

Новосибирск 2025 г.

СОДЕРЖАНИЕ

ЦЕЛЬ И ЗАДАЧИ	3
1 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ.....	4
2 ПОДГОТОВКА ФАЙЛОВ	7
3 ОПРЕДЕЛЕНИЕ ЗАВИСИМОСТЕЙ.....	8
4 НАПИСАНИЕ DOCKERFILE	9
5 ВЫВОД	11

ЦЕЛЬ И ЗАДАЧИ

Цель: Познакомиться с Docker, понять принцип его работы, собрать свой собственный образ и запустить его в контейнере.

Задачи:

1. Разработать приложение, реализующее простой API по протоколу http. Допускается взять уже существующее.
2. Приложение должно корректно обрабатывать сигналы остановки.
3. Написать для приложения Dockerfile для сборки образа контейнера.
4. Продемонстрировать работу приложения в контейнере. Для этого необходимо запустить контейнер и обеспечить доступ по сети.

ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Что такое докер?

Docker — это платформа для разработки, доставки и запуска приложений в контейнерах. Контейнеры позволяют упаковать приложение со всеми его зависимостями (библиотеками, настройками, средой выполнения) в стандартизированную единицу для разработки.

Ключевые особенности:

- **Изоляция:** приложения в контейнерах работают изолированно.
- **Переносимость:** контейнеры работают одинаково на любой системе.
- **Эффективность:** используют ресурсы оптимальнее виртуальных машин.
- **Повторяемость:** гарантируют идентичное окружение на всех этапах.

Основные компоненты:

- **Docker Engine:** ядро платформы.
- **Dockerfile:** инструкция для сборки образов.
- **Образы:** шаблоны для создания контейнеров.
- **Контейнеры:** запущенные экземпляры образов.

Начало работы с Docker:

Скачиваем официальный скрипт и запускаем его:

```
curl -fsSL https://get.docker.com -o get-docker.sh  
sudo sh get-docker.sh
```

Проверяем установку:

```
docker run hello-world
```

Порядок работы с Docker при создании собственного образа:

1. Подготовить файлы: подготовим программу, которую хотим запустить в контейнере.
2. Определить зависимости программы (операционная система, язык программирования, библиотеки и т.д.).

3. Написать Dockerfile.
4. Создать образ.
5. Запустить образ в контейнере.

Полезные команды:

Просмотр контейнеров: Список запущенных контейнеров:

```
docker ps
```

Ключи:

-a : вывод всех контейнеров (даже не запущенных)

-l : показать последний созданный контейнер

-q : показать только ID контейнеров

Запуск/остановка контейнера:

Остановка контейнера:

```
docker stop container_name_or_id
```

Принудительная остановка:

```
docker kill container_name_or_id
```

Запуск остановленного контейнера:

```
docker start container_name_or_id
```

Перезагрузка контейнера:

```
docker restart container_name_or_id
```

Пауза контейнера:

```
docker pause container_name_or_id
```

Возобновление контейнера:

```
docker unpause container_name_or_id
```

Запуск контейнера:

Запуск контейнера:

```
docker run image_name
```

Запуск в фоновом режиме (чтобы не занимать терминал):

```
docker run -d image_name
```

Запуск с именем (для удобства использования):

```
docker run --name my_container image_name
```

Проброс портов (отображение портов контейнера на порты устройства, где он запущен):

```
docker run -p 8080:80 image_name
```

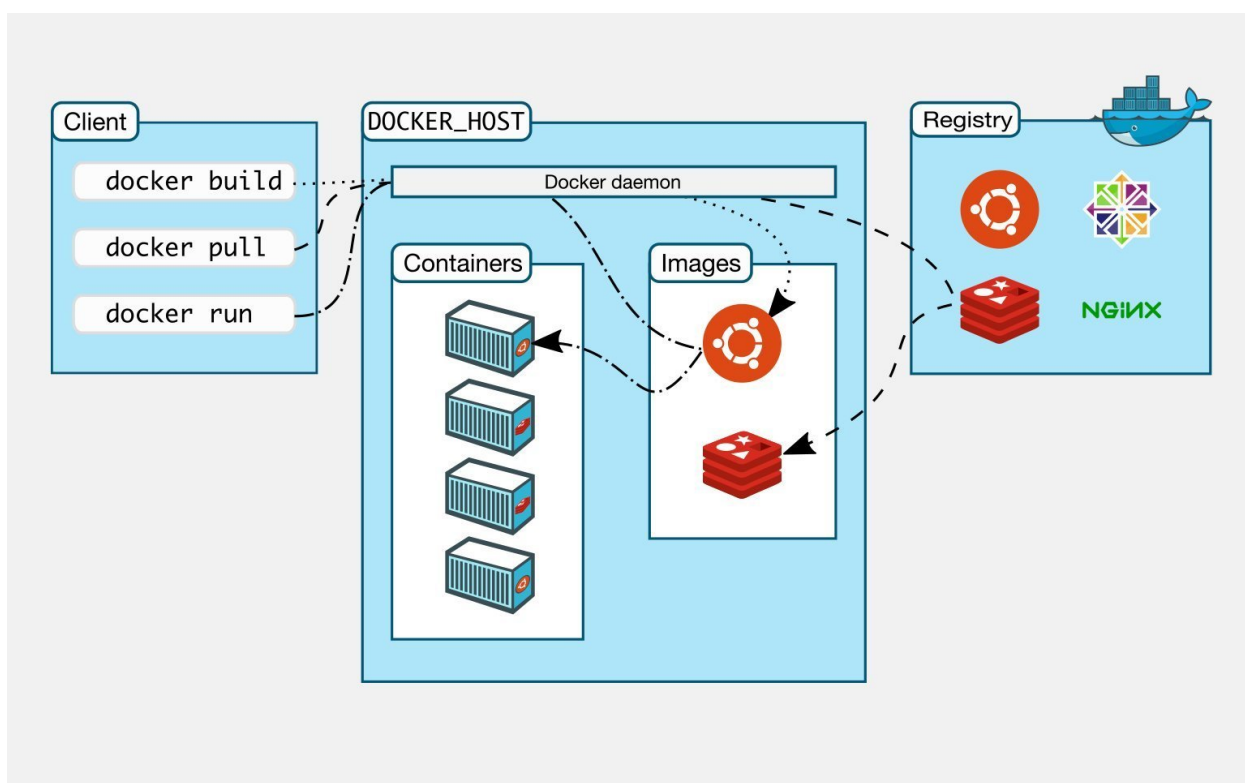
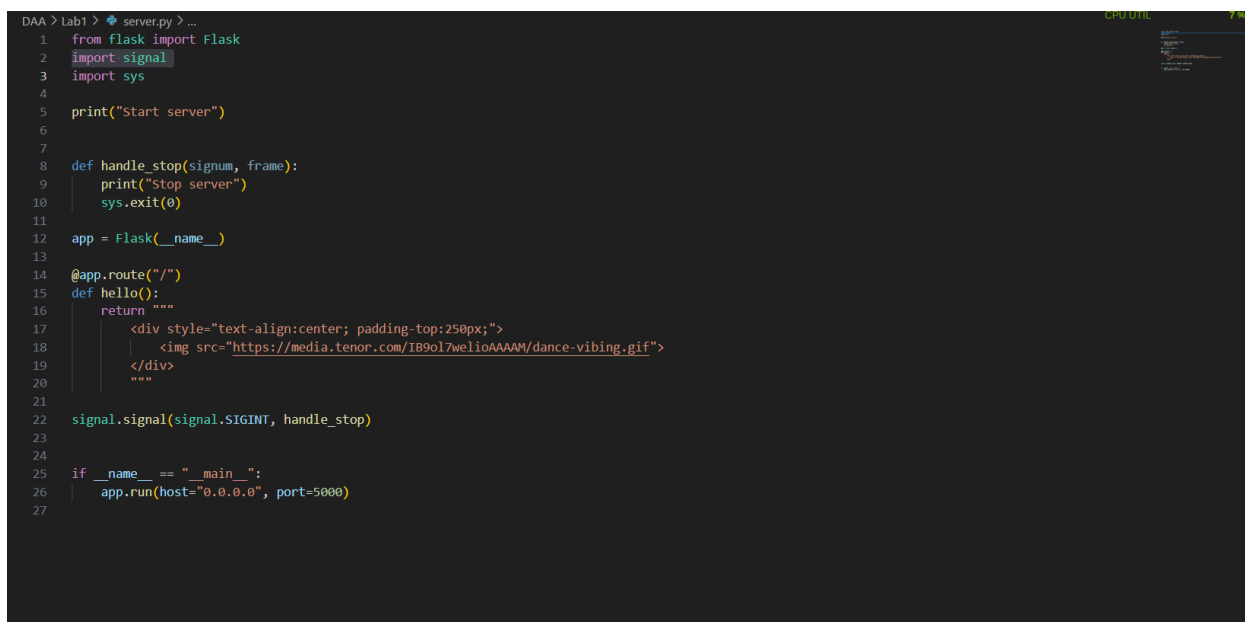


Рисунок 1 — Цикл работы Docker

ПОДГОТОВКА ФАЙЛОВ

В качестве программы, которую я буду контейнеризировать, будет выступать HTTP сервер на Python и flask. Сервер максимально примитивный и просто возвращает HTML документ. Помимо этого создан и привязан обработчик сигнала CTRL+C для корректного завершения работы

Реализация:



```
DAA > Lab1 > server.py > ...
1  from flask import Flask
2  import signal
3  import sys
4
5  print("Start server")
6
7
8  def handle_stop(signum, frame):
9      print("Stop server")
10     sys.exit(0)
11
12  app = Flask(__name__)
13
14  @app.route("/")
15  def hello():
16      return """
17          <div style="text-align:center; padding-top:250px;">
18              
19          </div>
20          """
21
22  signal.signal(signal.SIGINT, handle_stop)
23
24
25  if __name__ == "__main__":
26      app.run(host="0.0.0.0", port=5000)
27
```

Рисунок 2 — Пример реализации простого http сервера на Python

ОПРЕДЕЛЕНИЕ ЗАВИСИМОСТЕЙ

Операционная система: 22.0.4 (подойдет и другая версия и обязательно ubuntu, можно использовать alpine для экономии места)

Язык программирования: Python 3.12.3 (можно использовать и другую версию, главное, чтобы версия поддерживала flask)

Библиотеки, фреймворки: Flask 3.0.2 (подойдет и другая версия)

НАПИСАНИЕ DOCKERFILE

Что такое Dockerfile?

Dockerfile — это текстовый файл с инструкциями, который говорит Docker, как собрать образ контейнера.

Как написать Dockerfile?

Создается файл с названием Dockerfile (без расширения). Далее указываются инструкции для сборки образа. Их достаточно много, но я использовал самые базовые:

FROM

После ключевого слова FROM указывается базовый образ. В моем случае это операционная система Ubuntu 24.0.4

RUN

После ключевого слова RUN выполняются команды внутри контейнера во время сборки. Обычно используется для установки зависимостей, обновления пакетов, настройки системы. В моем случае после запуска образа Linux я выполняю на нем update, устанавливаю Python и Flask.

COPY

После ключевого слова COPY указываются файлы, которые будут перенесены с нашей системы на образ. В моем случае я хочу перенести файл server.py

CMD

После ключевого слова CMD указываются команды, которые должны выполняться при запуске образа. В моем случае нужно просто запустить файл server.py

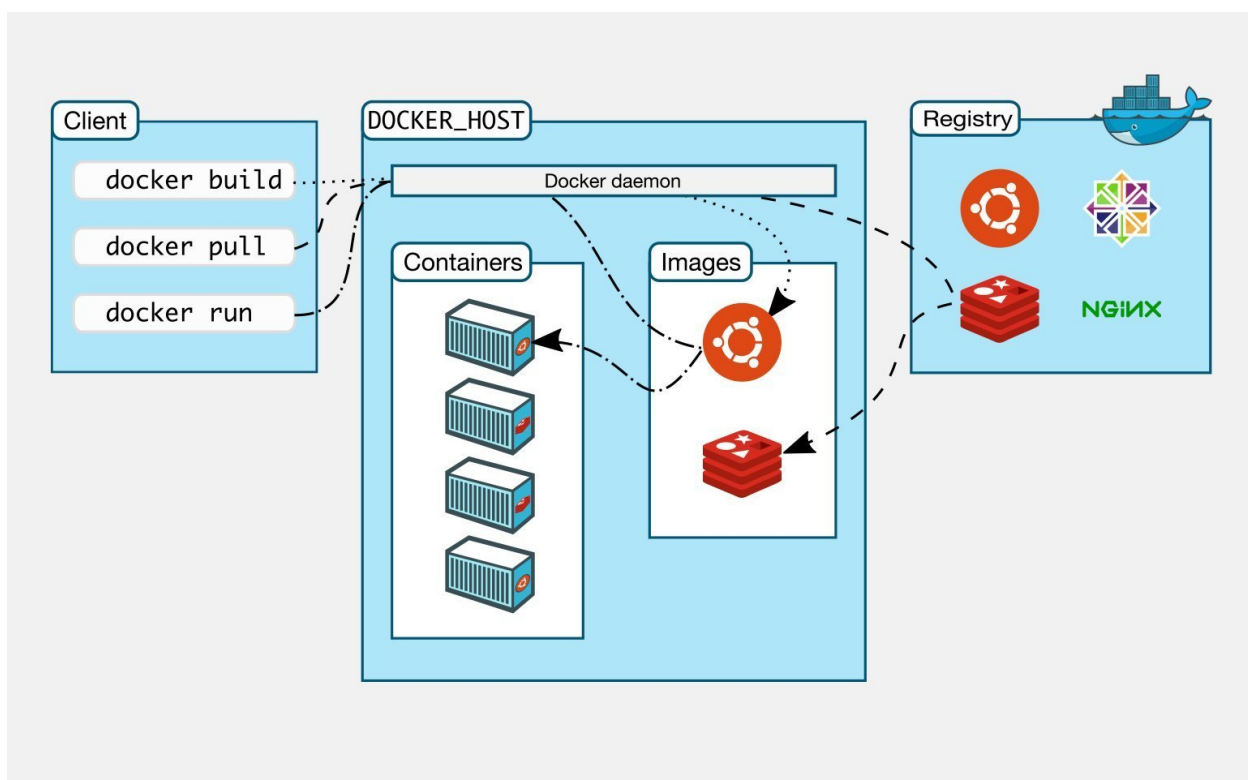


Рисунок 3 — Пример простого Dockerfile

ВЫВОД

В ходе работы я познакомился с новым инструментом Docker, изучил принципы его работы, команды его API и создал свой первый docker образ. Docker за последние годы стал очень популярен в мире IT, без него невозможно представить современную разработку, поэтому выполнение данной работы было очень полезным.